

EECS 376: Foundations of Computer Science

University of Michigan, Fall 2026

Solutions to Homework 1

Collaborators: None.

0. Before you start; before you submit

I read Handout 1 and the “Good Writing” section of Handout 2. I wrote the solutions below independently.

1. L^AT_EX (optional extra credit)

(a)

For (non-negative) functions $f(n)$ and $g(n)$, we say $f(n) = O(g(n))$ if there exist positive constants c, n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. The key features here are:

```
\begin{itemize}
  \item ``there exists a positive constant  $c$  ...  $f(n) \leq c \cdot g(n)$ ':
    this says that  $f(n)$  is \emph{upper bounded} by some \emph{constant multiple} of  $g(n)$ . That is,  $O$ -notation ``hides, ' or ignores, constant factors. Note also that it captures only an \emph{upper bound}:  $f(n)$  might actually be \emph{much smaller} than  $c \cdot g(n)$ , or not.
  \item ``there exists a positive constant  $n_0$  ... for all  $n \geq n_0$ ':
    this says that the upper bound holds for all  $n$  \emph{above some constant threshold}. It says nothing about the relationship below the threshold, and the notation hides the exact value of  $n_0$ .
\end{itemize}
```

(b)

The Fibonacci numbers may be defined by the recurrence relation

```
\[
F_n = F_{n-1} + F_{n-2}
\]
```

with base cases $F_0 = 0$ and $F_1 = 1$. We can compute the n -th Fibonacci number using the following recursive algorithm:

```
\begin{algorithmic}
\Require a natural number  $n$ 
\Ensure the  $n$ -th Fibonacci number
\Function{Fib}{ $n$ }
  \If{ $n=0$ } \State \Return 0 \EndIf
  \If{ $n=1$ } \State \Return 1 \EndIf
  \State \Return \Call{Fib}{ $n-1$ } + \Call{Fib}{ $n-2$ }
\EndFunction
\end{algorithmic}
```

2. Properties of logarithms

(b)

By D1, $c^{\log_c a} = a$ and $c^{\log_c b} = b$. Therefore, by P1,

$$b^{\log_c a} = \left(c^{\log_c b}\right)^{\log_c a} = c^{(\log_c b)(\log_c a)}.$$

Also $a = c^{\log_c a}$, so $\log_b a$ is the unique exponent t with $b^t = c^{\log_c a}$. The displayed identity shows that $t = \log_c a / \log_c b$ works; uniqueness follows from P2. Thus

$$\log_b a = \frac{\log_c a}{\log_c b}.$$

(c)

By D1, $b^{\log_b n} = n$. Hence, using P1,

$$a^{\log_b n} = \left(b^{\log_b a}\right)^{\log_b n} = b^{(\log_b a)(\log_b n)} = \left(b^{\log_b n}\right)^{\log_b a} = n^{\log_b a}.$$

(d)

Applying part (b) with $a = n$ gives

$$\frac{\log_b n}{\log_c n} = \frac{1}{\log_c b},$$

which depends only on b, c (and is defined for $n \neq 1$). Thus the ratio is a positive constant independent of n .

3. Welcome to EECS 376!

(a)

The print statement executes $\sum_{i=1}^n i = n(n+1)/2$ times. Each iteration has constant overhead, so the tight bound is $\Theta(n^2)$ (and hence also $O(n^2)$).

(b)

A binary representation of n with s bits satisfies $2^{s-1} \leq n < 2^s$. Thus $\Theta(n^2) = \Theta(4^s)$ as a worst-case function of s . This is exponential, not polynomial, so the algorithm is **not efficient**.

(c)

With s' decimal digits, $10^{s'-1} \leq n < 10^{s'}$, so the running time is $\Theta(100^{s'})$. This is exponential in s' , hence the algorithm is **not efficient**.

(d)

Since $\log n = \Theta(s)$ for binary input and $\log n = \Theta(s')$ for decimal input, algorithm A takes $\Theta(s)$ and $\Theta(s')$ time, respectively. Both are polynomial (indeed linear) in the input size, so A is efficient under either representation.

4. Practice with asymptotics

(a)

$f(n) = 3 \log_2 n$ and $g(n) = \frac{1}{8} \log_5 n$. By change of base, $f/g = 24 \log_2 5$, a positive constant. Therefore $f = O(g)$ and $f = \Omega(g)$, hence $f = \Theta(g)$.

(b)

Since $-1 \leq \sin n \leq 1$,

$$375n \leq f(n) \leq 377n.$$

Because $g(n) = 203n$, both f/g and g/f are bounded by positive constants. Thus $f = \Theta(g)$ (and both O and Ω statements hold).

(c)

$$\frac{f(n)}{g(n)} = n^3 \left(\frac{8}{9}\right)^n.$$

The ratio tends to 0: for $h(x) = 3 \ln x + x \ln(8/9)$, $h'(x) = 3/x + \ln(8/9) < 0$ for all sufficiently large x , and $h(x) \rightarrow -\infty$. Hence $f = O(g)$ and $f \neq \Omega(g)$. Therefore $f = O(g)$ only.

5. Prof. Upsilon and their oopsie claims

(a)

Necessarily true. For $n \geq 2$, $n \log n \leq n^2$, so $T_Y(n) = \Theta(n \log n) = O(n^2)$.

(b)

Not necessarily either. The given bounds concern maxima over inputs, not every input. The claim is true if every size- n input makes Z take n steps and Y take $n \log n$ steps. For a false example, for each n choose a special input u_n and let

$$T_Z(u_n) = n, \quad T_Y(u_n) = 1,$$

while on all other size- n inputs let $T_Z = n$ and $T_Y = n \log n$. The worst cases remain $\Theta(n)$ and $\Theta(n \log n)$, but on u_n , Z is slower than Y for every $n > 1$.

(c)

Not necessarily either. It is true when $T_X(n) = n^2$ and $T_Y(n) = n \log n$. It is false when $T_X(n) = n$ and $T_Y(n) = n \log n$, because then no input can make Y faster than X if their running times are input-independent.

(d)

Necessarily true. There are constants $c, C > 0$ such that, for all sufficiently large n , some size- n input u_n makes Y take at least $cn \log n$ steps, while every size- n input makes Z take at most Cn steps. For large enough n , $cn \log n > Cn$. On that particular u_n , therefore, Z runs faster than Y .

6. Set of proofs by contra-

(a)

Assume $A \subseteq B$ and $B \subseteq C$. For contradiction, suppose $A \not\subseteq C$. Then some $x \in A$ has $x \notin C$. Since $A \subseteq B$, $x \in B$; since $B \subseteq C$, $x \in C$, a contradiction. Therefore $A \subseteq C$.

(b)

The contrapositive is

$$x \notin \overline{A \cup B} \implies x \notin \overline{A \cap B}.$$

If $x \notin \overline{A \cup B}$, then $x \notin \overline{A}$ and $x \notin \overline{B}$. Hence $x \in A$ and $x \in B$, so $x \in A \cap B$, which is exactly $x \notin \overline{A \cap B}$.

7. Requirements for the potential method

(a)

$s_i = 2360/2^i \geq 0$, so it has a fixed lower bound. However, the decrease $s_i - s_{i+1} = 2360/2^{i+1}$ is not bounded below by a positive constant. The process nevertheless never reaches California in finitely many days, since $s_i > 0$ for every finite i ; no valid decreasing-potential proof of finite halting exists.

(b)

The proposed potential satisfies the lower-bound property because $s_i \geq 0$. It does *not* satisfy the decrease property. After 101 forced actions, one head remains. The “otherwise” rule then permits flipping that head together with one tail, leaving exactly one head again. Repeating this action forever is legal, so the potential stays at 1 and the process need not halt. Consequently no potential can prove that every allowed execution halts.

(c)

The number of misplaced elements need not decrease on every swap. Use instead the inversion count I , the number of pairs $j < k$ with $A[j] > A[k]$. It is nonnegative, and swapping any selected out-of-order pair decreases I by at least one (the selected pair is removed, and the net effect on pairs between the positions cannot increase the count enough to offset it). Therefore the process halts after at most I_0 swaps.

8. Euclid’s algorithm, extended

(a)

Since $x = qy + r$, set

$$\boxed{a = b', \quad b = a' - qb'}.$$

Indeed, $ax + by = b'(qy + r) + (a' - qb')y = a'y + b'r = g$.

(b)

The divisions are

$$376 = 1 \cdot 203 + 173, \quad 203 = 1 \cdot 173 + 30, \quad 173 = 5 \cdot 30 + 23,$$

$$30 = 1 \cdot 23 + 7, \quad 23 = 3 \cdot 7 + 2, \quad 7 = 3 \cdot 2 + 1, \quad 2 = 2 \cdot 1 + 0.$$

Using $a = b'$ and $b = a' - qb'$ gives the following trace.

i	x	y	q	r	a'	b'	a	b	s	s_{i+1}/s_i	$\leq 2/3$
0	376	203	1	173	75	-88	-88	163	579	376/579	Y
1	203	173	1	30	-13	75	75	-88	376	203/376	Y
2	173	30	5	23	10	-13	-13	75	203	53/203	Y
3	30	23	1	7	-3	10	10	-13	53	30/53	Y
4	23	7	3	2	1	-3	-3	10	30	9/30	Y
5	7	2	3	1	0	1	1	-3	9	3/9	Y
6	2	1	2	0	1	0	0	1	3	1/3	Y
7	1	0	-	-	-	-	1	0	1	-	-

The recursive base case returns $(g, a, b) = (1, 1, 0)$ for $(1, 0)$. Back-substitution gives

$$\begin{aligned}
1 &= 7 - 3 \cdot 2 = 7 - 3(23 - 3 \cdot 7) = 10 \cdot 7 - 3 \cdot 23 = 10(30 - 23) - 3 \cdot 23 = 10 \cdot 30 - 13 \cdot 23 \\
&= 10 \cdot 30 - 13(173 - 5 \cdot 30) = 75 \cdot 30 - 13 \cdot 173 = 75(203 - 173) - 13 \cdot 173 = 75 \cdot 203 - 88 \cdot 173 \\
&= 75 \cdot 203 - 88(376 - 203) = 163 \cdot 203 - 88 \cdot 376.
\end{aligned}$$

Thus the correct final output is $(g, a, b) = (1, -88, 163)$, and $376(-88) + 203(163) = 1$.